Table of Contents

Introduction **2**

Frontend item update **3**

ItemTable.tsx *3*

ItemGrid,tsx *3*

ItemUpdatePage.tsx *4*

Change the ItemForm.tsx: *6*

App.tsx *7*

Backend item update and test **8**

Remember the showTable or showGrid state **10**

Frontend item delete **11**

Change ItemListPage.tsx *11*

Change ItemTable.tsx *13*

Backend item delete and test **15**

Demo: Http Put and Delete

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini/Claude.

Introduction

This demo will guide you through the process of creating an item from the client side using HTTP PUT and HTTP DELETE request to a backend API.

Frontend Item Update (PUT):

- **Navigation & UI Setup:** Adds update navigation in ItemTable.tsx and ItemGrid.tsx using <Link> components.
- **Update Page Logic:** Implements ItemUpdatePage.tsx using React Router's useParams for route parameters, useQuery for fetching item data, and useMutation for HTTP PUT requests.
- **Form Reuse:** Updates ItemForm.tsx to handle both creation and editing by pre-populating fields via initialValues and handling loading/error states.

Backend Item Update:

- Implements HttpGet to retrieve the item and HttpPut to process the updated ItemDto inside ItemAPIController.

View Mode Persistence (localStorage):

- Persists the user's table/grid selection across visits in ItemListPage.tsx using lazy useState initialization to read from localStorage and useEffect with a [showTable] dependency array to save updates.

Frontend Item Delete (DELETE):

- **Page-Level Logic:** Configures useMutation in ItemListPage.tsx to dispatch HTTP DELETE requests and invalidate the ['items'] cache on success to auto-refresh the UI.
- **Component Action Triggers:** Passes an onItemDeleted callback prop to ItemTable.tsx and ItemGrid.tsx to trigger confirmation dialogs (window.confirm) via dedicated action <button>s.

Backend Item Delete & Testing:

- Implements the HttpDelete endpoint in ItemAPIController to delete items by ID and return 204 No Content.
- Verifies functionality via the UI and Scalar API testing interface.

Frontend item update

ItemTable.tsx

To add the frontend item update function and form, we start from adding the links.
In ItemTable.tsx, import a new component Link.
Then add an extra column “Actions” in the table header and add a Link in the table data.

```
ItemTable.tsx U
6-React-Intro > Demo-react-7-httpputdelete > myshop > src > items > Ite
1   import { useState } from 'react';
2   import { Table, Button } from 'react-bootstrap';
3   import { Item } from '../types/item';
4   import { Link } from 'react-router-dom';
26      <Table striped bordered hover>
27        <thead>
28          <tr>
29            <th>ID</th>
30            <th>Name</th>
31            <th>Price</th>
32            {showDescriptions && <th>Descriptions</th>}
33            {showImages && <th>Images</th>}
34            <th>Actions</th>
35          </tr>
36        </thead>
37        <tbody>
38          {items.map(item => (
39            <tr key={item.itemId}>
40              <td>{item.itemId}</td>
41              <td>{item.name}</td>
42              <td>{item.price} NOK</td>
43              {showDescriptions && <td>{item.description}</td>}
44              {showImages && (
45                  <td>{item.imageUrl ? (<img
46                    src={`${apiUrl}${item.imageUrl}`}
47                    alt={item.name}
48                    width="120"
49                    onError={(e) => (e.currentTarget.style.display = 'none')}
50                  />) : (<span>No Image</span>
51                  )}</td>)}
52                <td className="text-center">
53                  <Link to={`/itemupdate/${item.itemId}`}>Update</Link>
54                </td>
55            </tr>
56          ))}
57        </tbody>
58      </Table>
```

ItemGrid,tsx

In ItemGrid,tsx, import a new component Link.

```
ItemGrid.tsx U
6-React-Intro > Demo-react-7-httpputdelete > myshop > src > items > Iten
1   import { Card, Col, Row } from 'react-bootstrap';
2   import { Item } from '../types/item';
3   import { Link } from 'react-router-dom';
```

In the **Card** component add a ** ** for the **Button** (we use Link but it will be rendered as a button. There will be a Button for delete later, so we have the **de-flex** class from Bootstrap). The changed **className** for the **Card.Body** and for the ** ** that contains the Button, is to ensure that the Button will stick to the bottom of the card, so that it does not float up and down because of different description length.

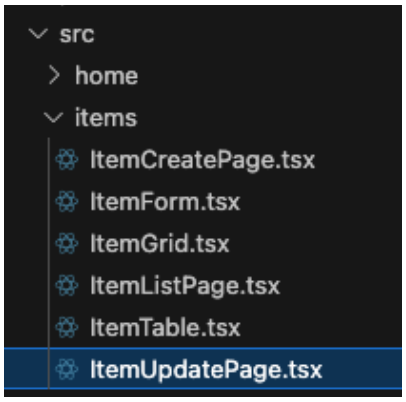
In web applications, choosing between <Link> and <button> comes down to **intent and semantically correct HTML**:

- **<Link> (Navigation / GET):** Used to change the URL, navigate to a new page/view, or load a new resource without altering data on the server. Semantically, it renders an anchor <a> tag, which browsers and screen readers understand as a link.
- **<button> (Action / POST, PUT, DELETE):** Used to trigger an inline action, submit a form, or mutate data (such as opening a modal, toggling a state, or executing a delete call).

Even if a link is styled visually to look like a button (e.g., using Bootstrap's btn classes), using a <Link> ensures proper browser behaviors such as allowing users to right-click to open in a new tab, supporting native bookmarking, and giving screen readers accurate semantic information.

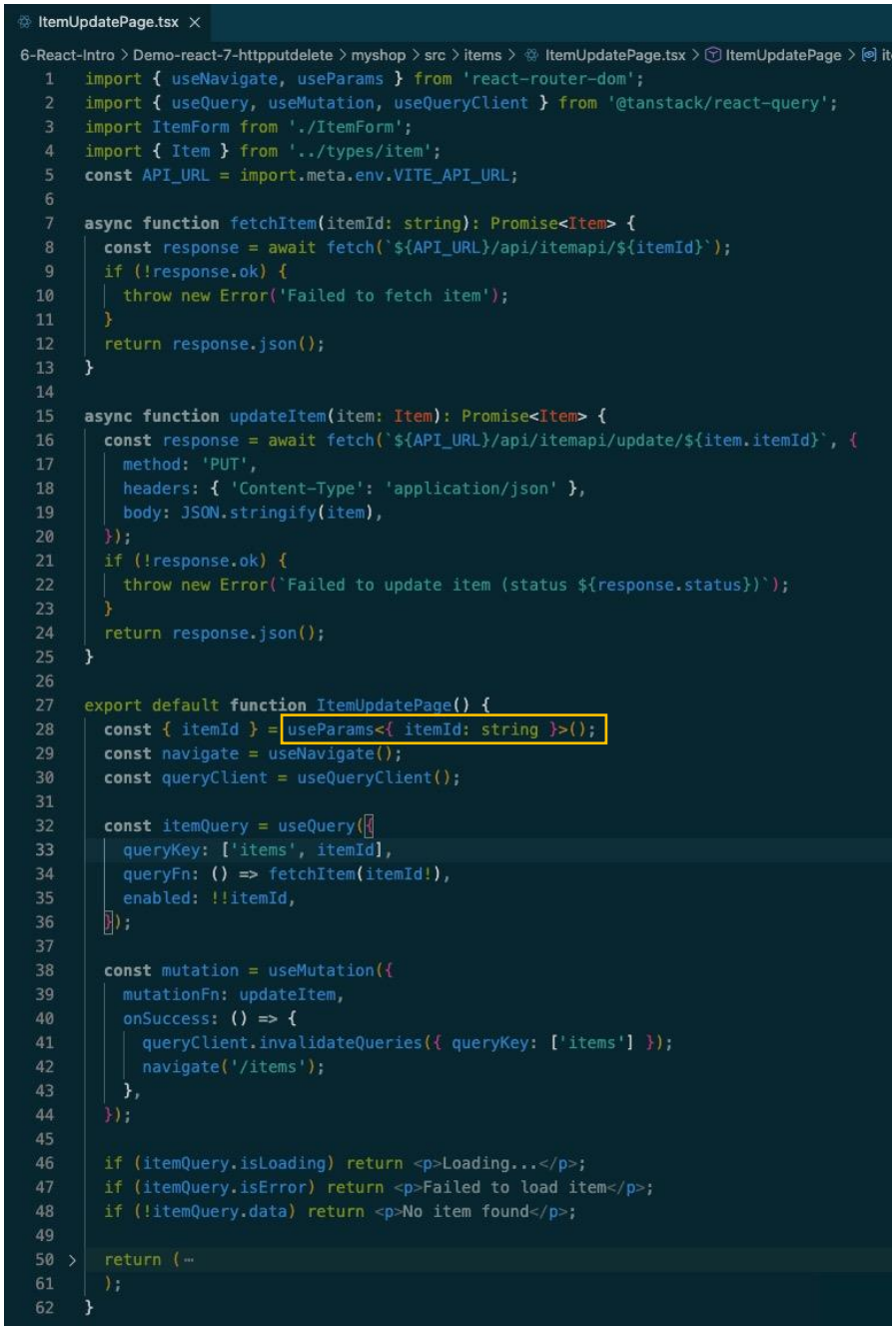
```
17              <Card className="h-100">
18                {item.imageUrl ? (
19                  <Card.Img
20                    variant="top"
21                    src={`${apiUrl}${item.imageUrl}`}
22                    alt={item.name}
23                    onError={(e) => (e.currentTarget.style.display = 'none')}
24                  />
25                ) : (
26                  <div style={{ textAlign: 'center', padding: '1rem' }}>No Image</div>
27                )}
28                <Card.Body className="d-flex flex-column">
29                  <Card.Title>{item.name}</Card.Title>
30                  <Card.Text>
31                    {item.description}
32                  </Card.Text>
33                  <Card.Text>
34                    Price: {item.price} NOK
35                  </Card.Text>
36                  <div className="mt-auto d-flex justify-content-between gap-2">
37                    <Link to={`/itemupdate/${item.itemId}`} className="btn btn-primary mt-3">Update</Link>
38                  </div>
39                </Card.Body>
40              </Card>
```

Now we add a new file under src/items.



ItemUpdatePage.tsx

The ItemUpdatePage.tsx look like this:



It has 5 parts: **imports**, the **query** for fetching the existing item, the **mutation** for handling the update, **conditional rendering** for loading/error/not-found states, and the **returned `ItemForm`** filled with the item's data.

The `itemId` is obtained from the URL using `useParams`. This ID is used to fetch the item data from the backend. It isn't passed as a prop but is extracted directly from the URL, which simplifies `ItemUpdatePage` since it takes no input properties. Note the property name needs to align with the route parameter name declared in `App.tsx` (e.g. `/itemupdate/:itemId`).

Fetching the item uses `useQuery`. This is consistent with how `ItemCreatePage` already uses React Query for its mutation, and it means loading and error states (`itemQuery.isLoading`, `itemQuery.isError`) come for free instead of being tracked by hand. The `queryKey`,

`['items', itemId]`, identifies this particular item in the cache, distinct from the `['items']` key used for the full list. The `enabled: !!itemId` option prevents the query from firing before `itemId` is available from the URL params.

Updating the item uses `useMutation`, exactly as `ItemCreatePage` does for creation. The `updateItem` function performs an Http Put request to `${API_URL}/api/itemapi/update/${item.itemId}`, which is very similar to the Http Post used for creation, except for the method and the target URL. On success, `onSuccess` invalidates both the item list and the specific item's cache entry (since `['items']` invalidation also matches any query keyed with `['items', itemId]`, as React Query treats query keys hierarchically), then navigates back to `/items`.

The `ItemForm` is passed `itemId` and `initialValues` from the fetched item, so the form fields are pre-populated for editing, along with `error` and `submitting` from the mutation — the same pattern used in `ItemCreatePage`. Note there is no separate `isUpdate` flag: `ItemForm` already infers update-vs-create mode from whether `itemId` is defined, both for the submit button's label and for the `item` object built on submit.

Change the ItemForm.tsx:

ItemForm.tsx U

6-React-Intro > Demo-react-7-httpputdelete > myshop > src > items > ItemForm.tsx > ItemForm

```tsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Form, Button, Alert } from 'react-bootstrap';
import { Item } from '../types/item';

interface ItemFormProps {
  onItemChanged: (newItem: Item) => Promise<unknown> | void;
  itemId?: number;
  initialValues?: Partial<Item>;
  error?: string | null;
  submitting?: boolean;
}

export default function ItemForm({
  onItemChanged,
  itemId,
  initialValues,
  error = null,
  submitting = false,
}: ItemFormProps) {
  const [name, setName] = useState<string>(initialValues?.name ?? '');
  const [price, setPrice] = useState<number>(initialValues?.price ?? 0);
  const [description, setDescription] = useState<string>(initialValues?.
  description ?? '');
  const [imageUrl, setImageUrl] = useState<string>(initialValues?.imageUrl ?? '');
  const navigate = useNavigate();

  const onCancel = () => {
    navigate(-1); // This will navigate back one step in the history
  };

  const handleSubmit = async (event: React.SyntheticEvent<HTMLFormElement>) => {
    event.preventDefault();
    const item: Item = { itemId, name, price, description, imageUrl };
    await onItemChanged(item);
  };


  return (…
  );
}
```

The component's props are unchanged from the version used by `ItemCreatePage`.
No new properties are needed to support updating, since `initialValues`, `error`, and `submitting` were already part of `ItemFormProps` and simply get reused here with different values. The default values of each field's state are set from `initialValues` if provided (populating the form when editing), otherwise falling back to the same empty defaults as item creation.

One correction from the previous version: `ItemForm` now destructures `error` and `submitting` from its props (with sensible defaults) and use them directly for the `Alert` and the submit button's disabled/label state. It no longer keeps its own local `error` or `submitting` state, since API success/failure is now entirely tracked by the parent page's `useMutation`, and `handleSubmit` no longer needs a `try/catch`. It simply awaits `onItemChanged(item)` and lets any rejection propagate to the mutation.

```
89        <Button variant="primary" type="submit" disabled={submitting}>
90          {submitting ? 'Saving...' : itemId ? 'Update Item' : 'Create Item'}
91        </Button>
92        <Button variant="secondary" onClick={onCancel} className="ms-2" disabled=
          {submitting}>
93          Cancel
94        </Button>
```

The form is only changed in that the submit update is conditional rendered to update item or create item depending of if `itemId` is null or not.

App.tsx

Add route entry to itemupdate/:itemId

App.tsx U

6-React-Intro > Demo-react-7-httpputdelete > myshop > src > App.tsx > App

```tsx
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import Container from 'react-bootstrap/Container'
import HomePage from './home/HomePage'
import ItemListPage from './items/ItemListPage'
import ItemCreatePage from './items/ItemCreatePage'
import ItemUpdatePage from './items/ItemUpdatePage'
import NavMenu from './shared/NavMenu'
import './App.css'

const queryClient = new QueryClient() // creates the single instance

export default function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Container>
        <NavMenu />
        <Router>
          <Routes>
            <Route path="/" element={<HomePage />} />
            <Route path="/items" element={<ItemListPage />} />
            <Route path="/itemcreate" element={<ItemCreatePage />} />
            <Route path="/itemupdate/:itemId" element={<ItemUpdatePage />} />
            <Route path="*" element={<Navigate to="/" replace />} />
          </Routes>
        </Router>
      </Container>
    </QueryClientProvider>
  )
}
```

Backend item update and test

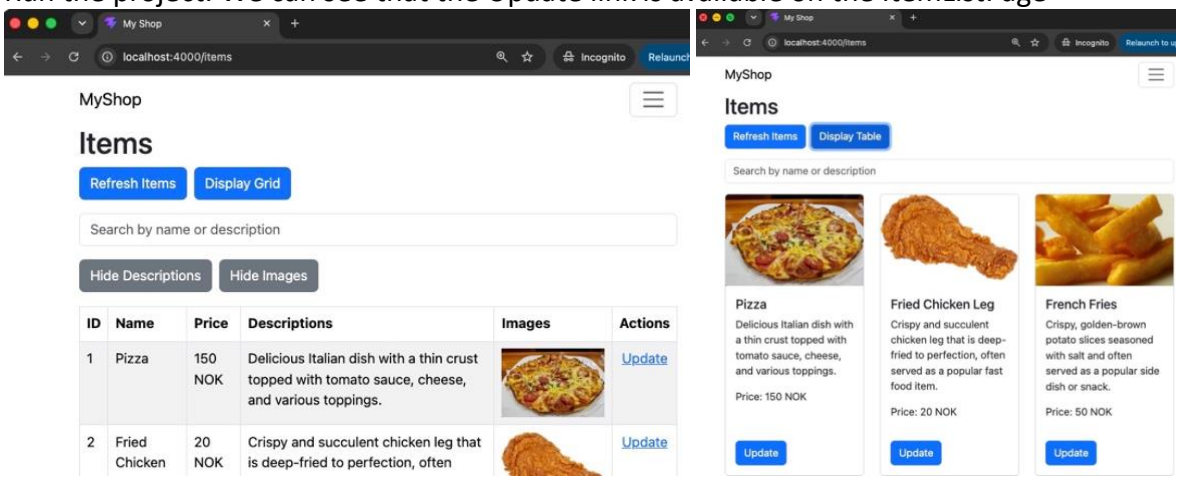
We need two methods in **ItemAPIController**. One **HttpGet** that retrieves the item to be updated:

```
65      [HttpGet("{id}")]
        0 references
66      public async Task<IActionResult> GetItem(int id)
67      {
68          var item = await _itemRepository.GetItemById(id);
69          if (item == null)
70          {
71              _logger.LogError("[ItemAPIController] Item not found for the ItemId {ItemId:0000}", id);
72              return NotFound("Item not found for the ItemId");
73          }
74          return Ok(item);
75      }
```

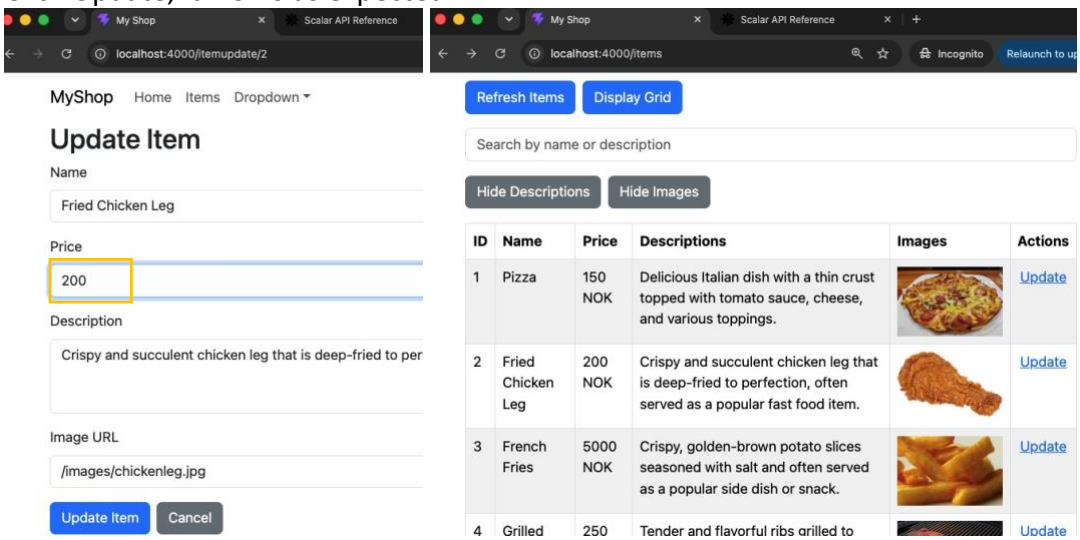
Another **HttpPut** method that updates the item with **ItemDto** from the Item Form.

```
77      [HttpPut("update/{id}")]
        0 references
78      public async Task<IActionResult> Update(int id, [FromBody] ItemDto itemDto)
79      {
80          if (itemDto == null)
81          {
82              return BadRequest("Item data cannot be null");
83          }
84          // Find the item in the database
85          var existingItem = await _itemRepository.GetItemById(id);
86          if (existingItem == null)
87          {
88              return NotFound("Item not found");
89          }
90          // Update the item properties
91          existingItem.Name = itemDto.Name;
92          existingItem.Price = itemDto.Price;
93          existingItem.Description = itemDto.Description;
94          existingItem.ImageUrl = itemDto.ImageUrl;
95          // Save the changes
96          bool updateSuccessful = await _itemRepository.Update(existingItem);
97          if (updateSuccessful)
98          {
99              return Ok(existingItem); // Return the updated item
100         }
101
102         _logger.LogWarning("[ItemAPIController] Item update failed {@item}", existingItem);
103         return StatusCode(500, "Internal server error");
104     }
```

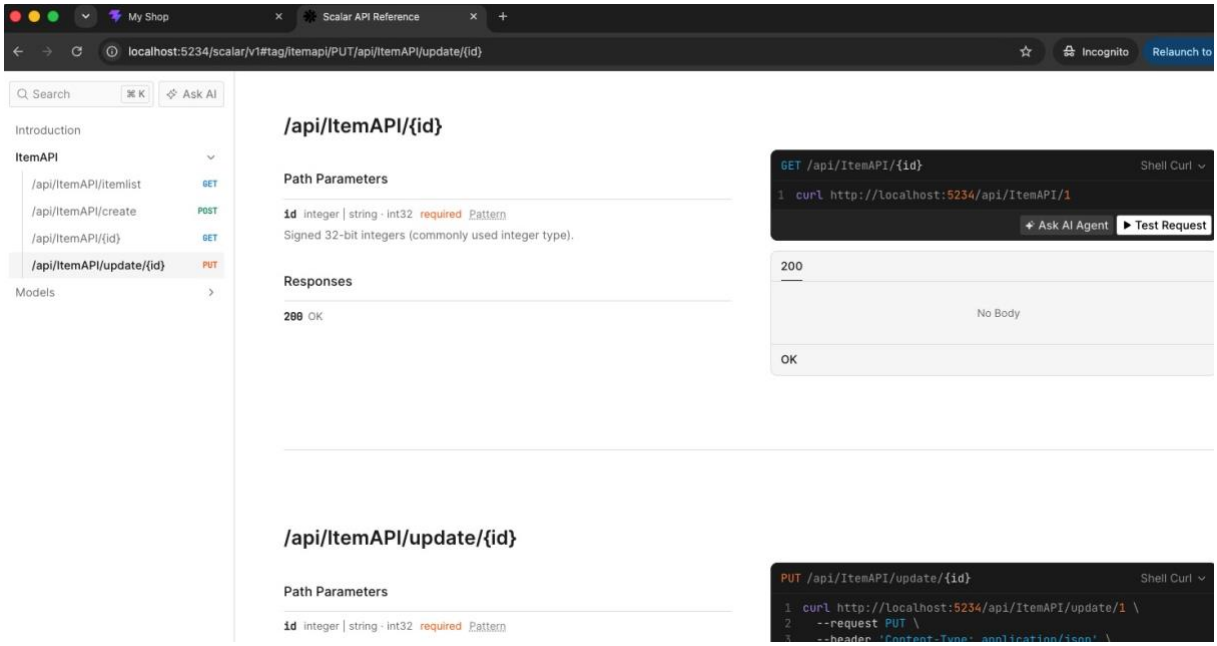
Run the project. We can see that the Update link is available on the ItemListPage



Click Update, it works as expected.



The Scalar UI also has two more methods:

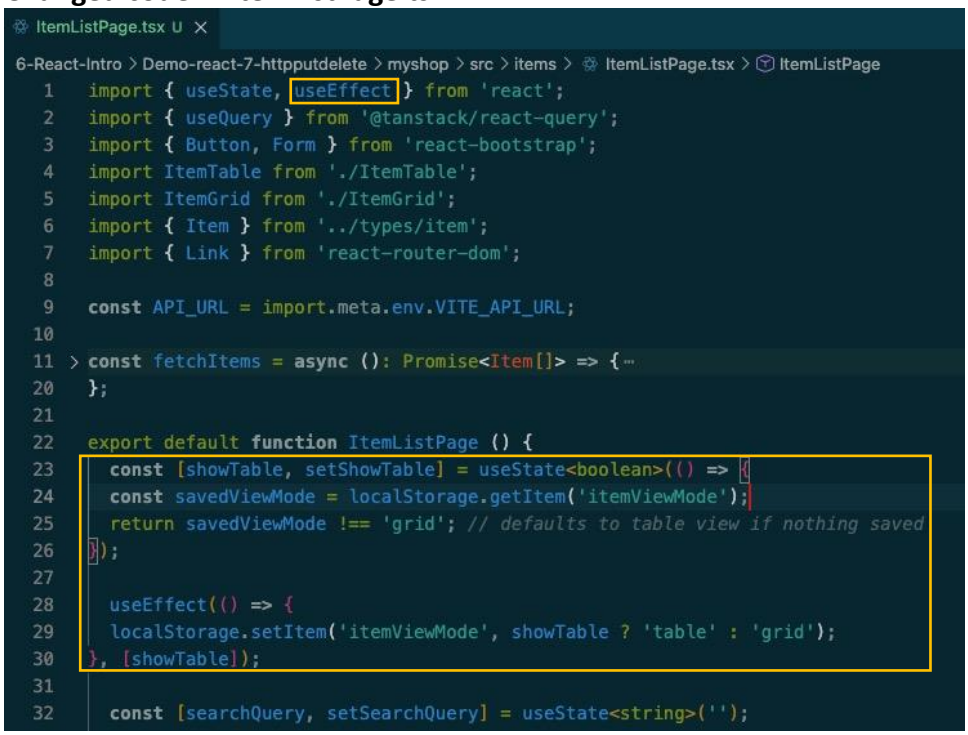


Remember the showTable or showGrid state

You may notice that whenever the user comes back to `ItemListPage`, it displays `ItemTable`, no matter what view mode was previously selected. To remember the view mode across visits, we use the browser's `localStorage` to persist it, and read it back when the component initialises.

We read the saved view mode when initialising the `showTable` state, using React's lazy initial state pattern: passing a function to `useState` instead of a plain value. This function runs exactly once, on the component's first render, before anything is painted to the screen.

Changed code in ItemListPage.tsx:



- `useState<boolean>(() => {...})`: the function passed here reads `itemViewMode` from `localStorage`. If the saved value is `'grid'`, `showTable` initializes to `false`; otherwise (including when nothing has been saved yet) it initializes to `true`, matching the original default.
- `useEffect`**:** a commonly used hook function that performs side effects (like updating `localStorage`) in response to component state changes; it runs once after the initial render and re-triggers whenever any value in its dependency array changes.
- The `useEffect`, with `[showTable]` as its dependency array, runs whenever `showTable` changes (i.e., whenever the user toggles between table and grid view) and saves the current mode back to `localStorage` as `'table'` or `'grid'`, so the next visit can read it back correctly.

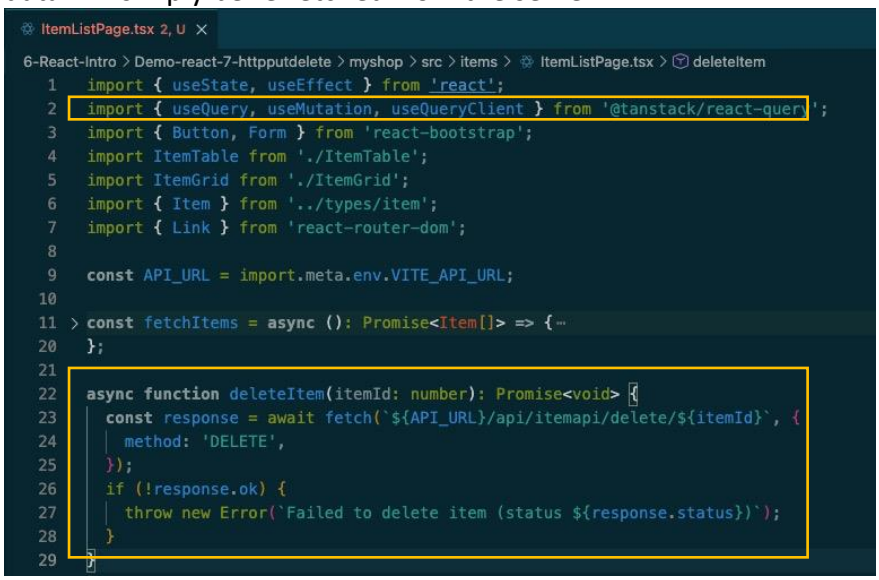
You can test it. Now the view mode is remembered.

Frontend item delete

We are going to add the item delete function and Http Delete request. We need to update `ItemListPage`, `ItemTable`, and `ItemGrid`.

Change ItemListPage.tsx

Since item deletion, like creation and updating, is a mutation against the server, we handle it with React Query's `useMutation`, consistent with the rest of the app.
A standalone `deleteItem` function performs the actual Http Delete request to `${API_URL}/api/itemapi/delete/${itemId}`, and throws an error if the response is not OK, following the same pattern used by `createItem` and `updateItem`.
`useMutation` is configured with `deleteItem` as its `mutationFn`. Its `onSuccess` callback invalidates the `['items']` query, so the list refetches and the deleted item disappears from the UI. There's no need to manually filter the deleted item out of a local array, since the data will simply be re-fetched from the server.



`handleItemDeleted` itself becomes a thin wrapper: it takes `itemId` as input, pops up a confirmation dialog with `window.confirm`. If confirmed, it calls `deleteMutation.mutate(itemId)`. Unlike `handleItemCreated`/`handleItemUpdated`, it doesn't need to be an `async` function or use `try/catch`, since `useMutation` handles the request lifecycle (including errors) internally; we're not awaiting a result or navigating afterward, just firing the delete and letting the cache invalidation update the screen.

```tsx
export default function ItemListPage () {
  const [showTable, setShowTable] = useState<boolean>(() => {
  const savedViewMode = localStorage.getItem('itemViewMode');
  return savedViewMode !== 'grid'; // defaults to table view if nothing saved
});

  useEffect(() => {…
}, [showTable]);

  const [searchQuery, setSearchQuery] = useState<string>('');

  const toggleTableOrGrid = () => setShowTable(prev => !prev);

  const { data: items = [], isLoading, error, refetch } = useQuery<Item[], Error>({
    queryKey: ['items'],
    queryFn: fetchItems,
  });

  const queryClient = useQueryClient();

  const deleteMutation = useMutation({
    mutationFn: deleteItem,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['items'] });
    },
  });

  const handleItemDeleted = (itemId: number) => {
    const confirmDelete = window.confirm(`Are you sure you want to delete the item ${itemId}?`);
    if (confirmDelete) {
      deleteMutation.mutate(itemId);
    }
  };

  const filteredItems = items.filter(item =>
    item.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
    (item.description ?? '').toLowerCase().includes(searchQuery.toLowerCase())
  );

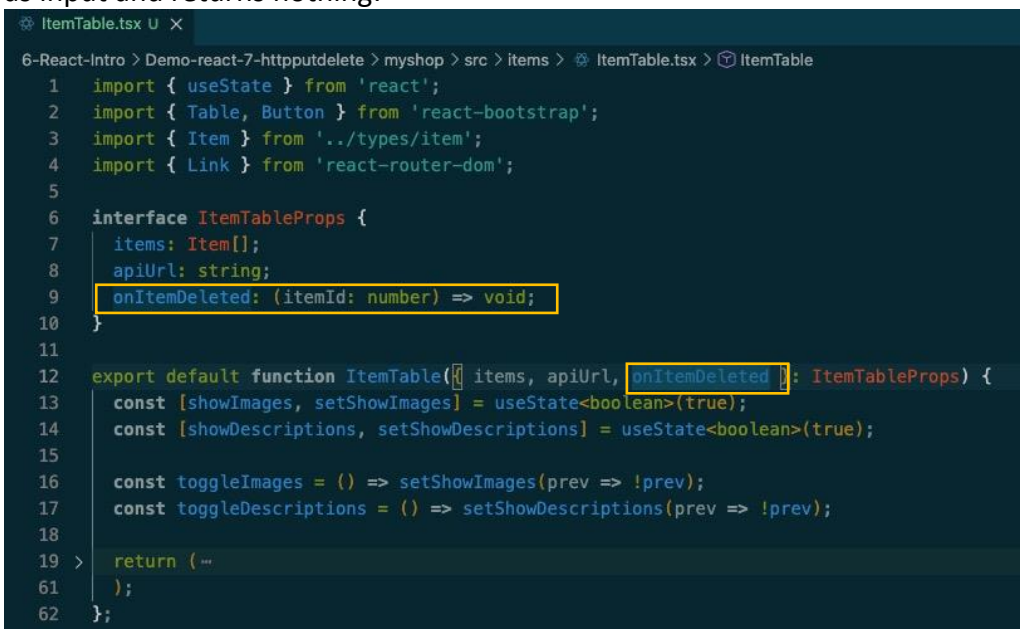
  return (…
  );
};
```

```tsx
  return (
    <div className="text-start">
      <h1>Items</h1>
      <Button onClick={() => refetch()} variant="primary" className="mb-3 me-2" disabled={isLoading}
      >
        {isLoading ? 'Loading...' : 'Refresh Items'}
      </Button>
      <Button onClick={toggleTableOrGrid} variant="primary" className="mb-3 me-2">
        {showTable ? `Display Grid` : 'Display Table'}
      </Button>
      <Form.Group className="mb-3">
      <Form.Control
        type="text"
        placeholder="Search by name or description"
        value={searchQuery}
        onChange={e => setSearchQuery(e.target.value)}
      />
      </Form.Group>
      {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
      {showTable
        ? <ItemTable items={filteredItems} apiUrl={API_URL} onItemDeleted={handleItemDeleted} />
        : <ItemGrid items={filteredItems} apiUrl={API_URL} onItemDeleted={handleItemDeleted} />}
      <Link to="/itemcreate" className="btn btn-secondary mt-3">Add New Item</Link> {/* this is a
      button */}
    </div>
  );
```

In the returned component, `handleItemDeleted` is passed down to `ItemTable` and `ItemGrid` as the `onItemDeleted` prop. It's defined here in `ItemListPage`, rather than inside `ItemTable`/`ItemGrid` themselves, because it represents page-level logic, deciding *what* happens when an item is deleted, while the table and grid components are only responsible for triggering that action, not deciding its effect.

Change ItemTable.tsx

In `ItemTable.tsx`, we add an input property, `onItemDeleted`, a function that takes `itemId` as input and returns nothing:

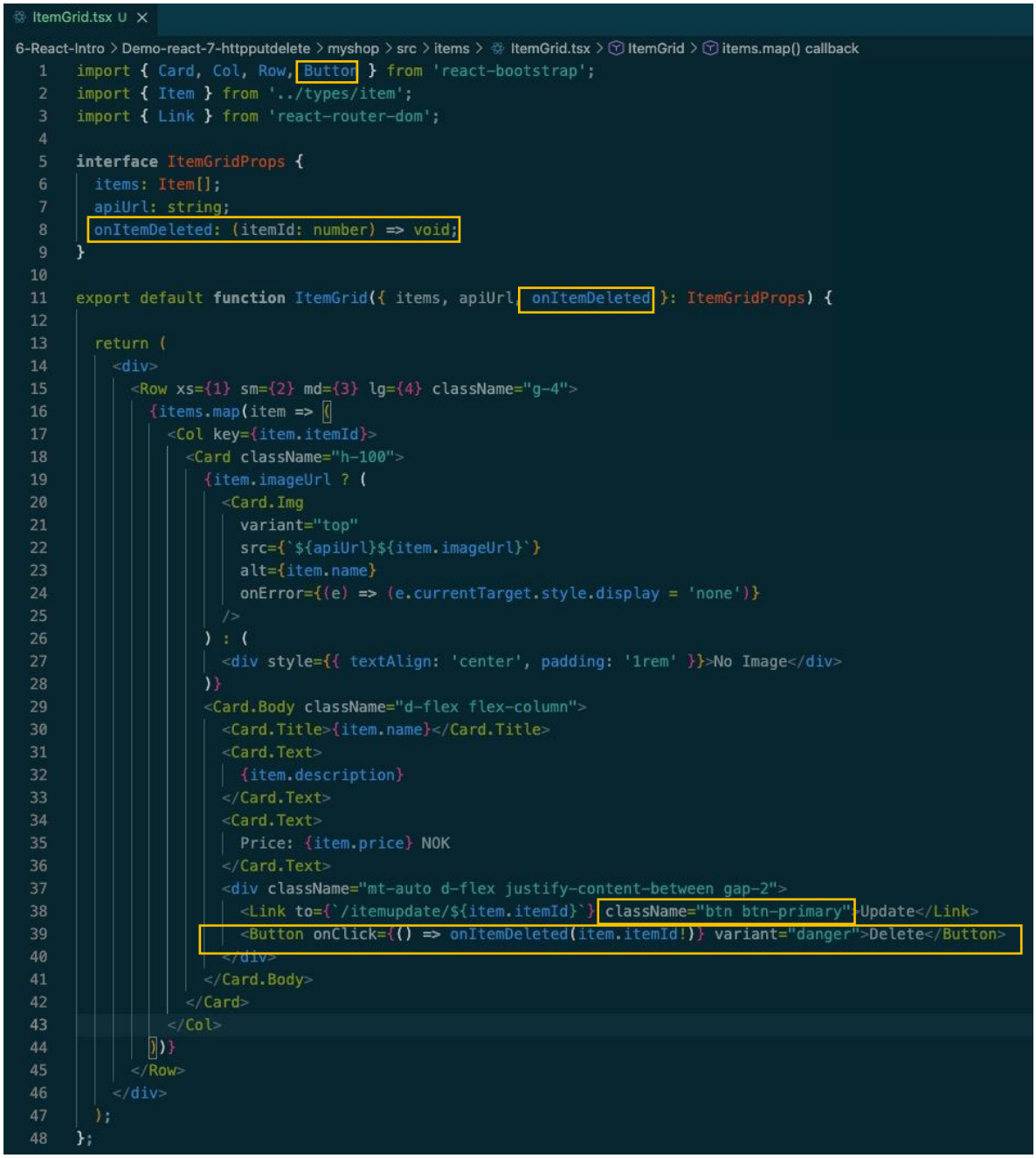


In the returned component, a Button triggers `onItemDeleted`, though it looks like a link. Here we adhere to our principles of using Link for navigating to some pages, and Button for triggering actions.

In the `onClick` handler, we use `item.itemId!`, the non-null assertion operator. This is appropriate here because, in `ItemTable`, items are retrieved from the database, which never returns a null `itemId`, but the `Item` type marks `itemId` as optional (to accommodate new, unsaved items in `ItemForm`). If there is no !, TypeScript would flag a type mismatch between the possibly `undefined` `item.itemId` and `onItemDeleted`'s parameter, which only accepts a `number`.



Similarly, in `ItemGrid.tsx`, an `onItemDeleted` property is added, along with a `Button` styled in red (`variant="danger"`) that triggers it. We display `item.itemId` in the card title for convenience, and `gap-2` on the button container ensures spacing between the Update and Delete buttons. The Update Link is also changed in className to make the height of the two Buttons match.

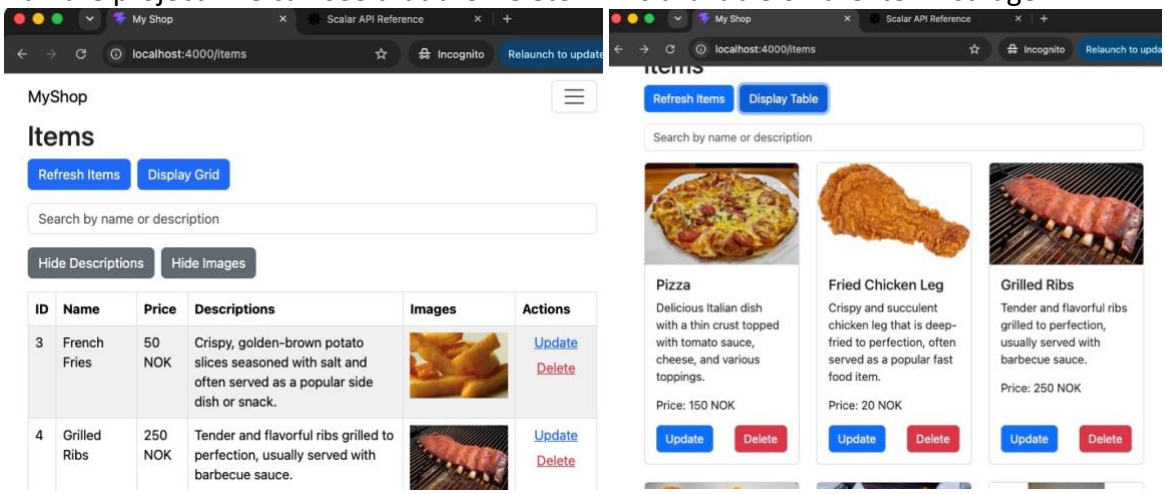


Backend item delete and test

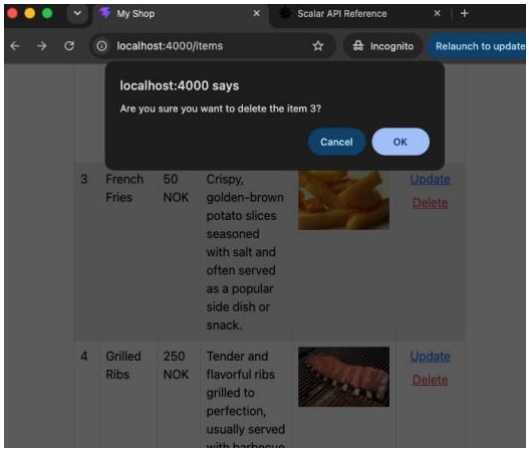
We add a **DeleteConfirmed** method to the **ItemAPIController**. It calls the item repository to delete the item with the id, and returns no content if success.

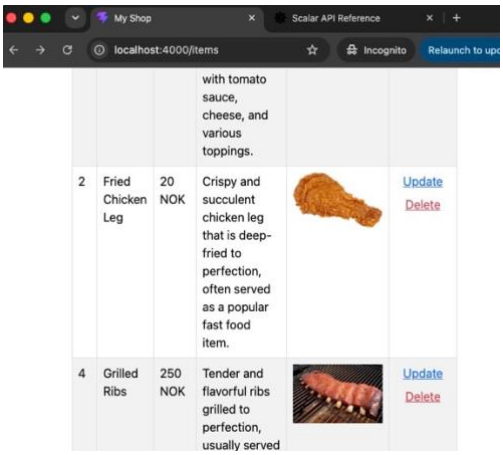
```
106     [HttpDelete("delete/{id}")]
        0 references
107     public async Task<IActionResult> DeleteConfirmed(int id)
108     {
109         bool returnOk = await _itemRepository.Delete(id);
110         if (!returnOk)
111         {
112             _logger.LogError("[ItemAPIController] Item deletion failed for the ItemId {ItemId:0000}", id);
113             return BadRequest("Item deletion failed");
114         }
115         return NoContent(); // 200 Ok is commonly used when the server returns a response body with additional
            information about the result of the request. For a DELETE operation, there's generally no need to return
            additional data, making 204 NoContent a better fit.
116     }
```

Run the project. We can see that the Delete link is available on the ItemListPage

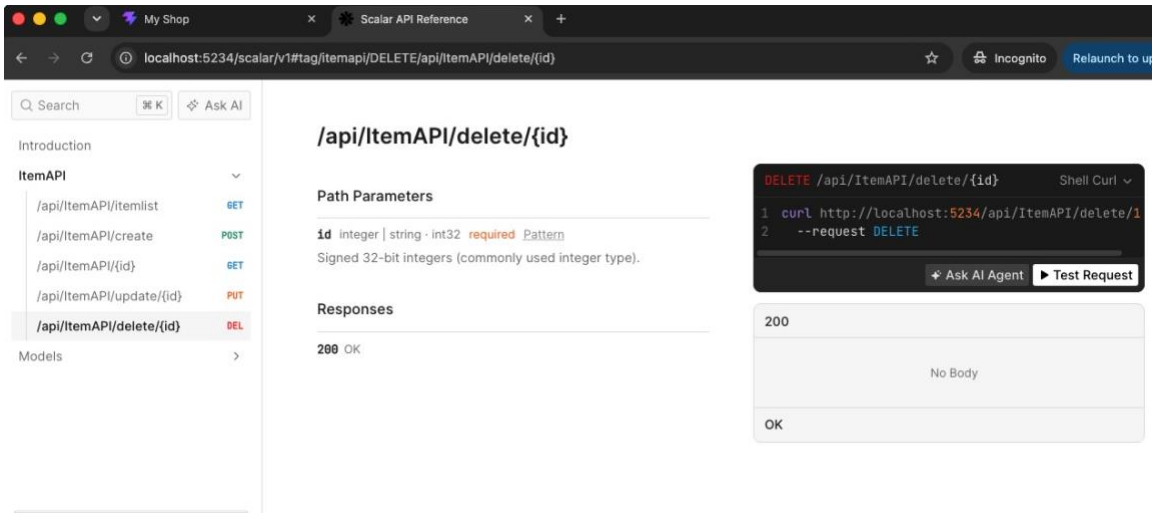


Click to delete one item (3), it is deleted.





The Scalar UI also shows an extra delete method.



Congratulation! You have accomplished item update and delete! ☺